

Proof-Carrying Streamed Workflow Graphs

Preserving Completion Viability
with Evolving Certificates

CASE STUDY / OPUS WORK KNOWLEDGE GRAPH



Phillip Kingston · Théo Fagnoni AppliedAI, United Arab Emirates

1st Conference and Workshop on Mathematics and Artificial Intelligence
Nazarbayev University, Astana · 9–11 September 2026

Roadmap

01	Motivation	Workflow construction and dead ends
02	Viability and residuals	Completion viability and residual updates
03	Certificates	Certificate meaning and checked updates
04	Viability guarantee	Viability after every admitted prefix
05	Proof states	Encoding, updates, and the distinctions they must retain

Workflow construction must preserve a way to finish

REUSABLE WORK

A shared graph

A **Work Knowledge Graph (WKG)** organizes reusable tasks and the execution flow between them.

Workflows are assembled from this shared graph for a specific business context.

INCREMENTAL CONSTRUCTION

One addition at a time

AI systems can construct workflow graphs while the workflow is still unfinished.

A locally valid addition can make the remaining requirements impossible to satisfy.

An **acceptable completion** is a finished workflow satisfying all requirements of the policy, including required evidence and authorizations.

Opus WKG supplies reusable pieces of executable work

01

Nodes

Single-responsibility tasks that can be assigned, executed, observed, and reused.

02

Directed edges

Execution handoffs: one task's output can become another task's input or trigger.

03

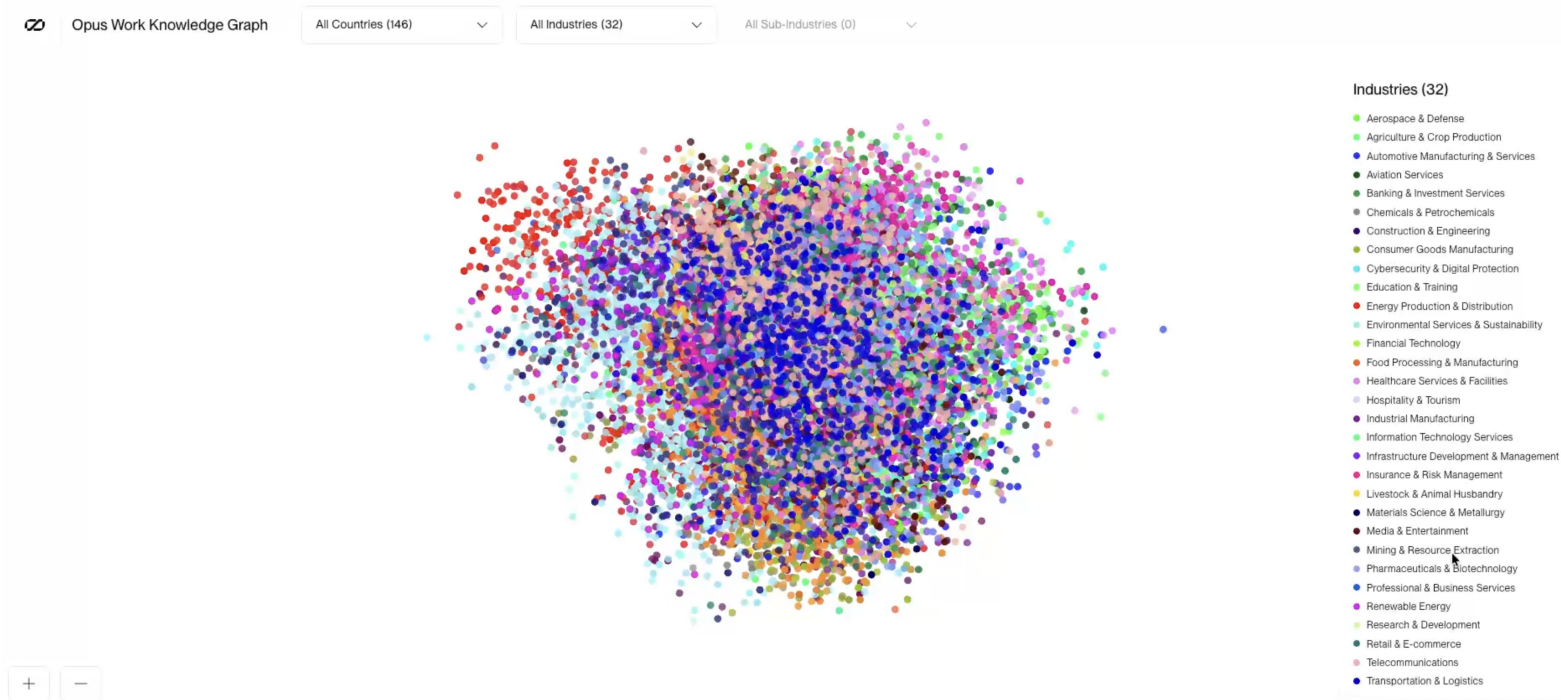
Workflows

Directed subgraphs assembled from shared tasks and handoffs for a particular context.

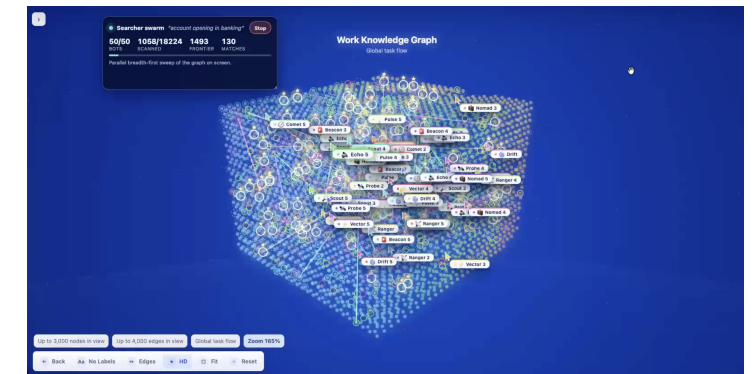
Reusing tasks from the WKG does not remove the evidence and authorization requirements of the workflow being built.

The WKG organizes work for reuse across workflows

WORK ACROSS INDUSTRIES



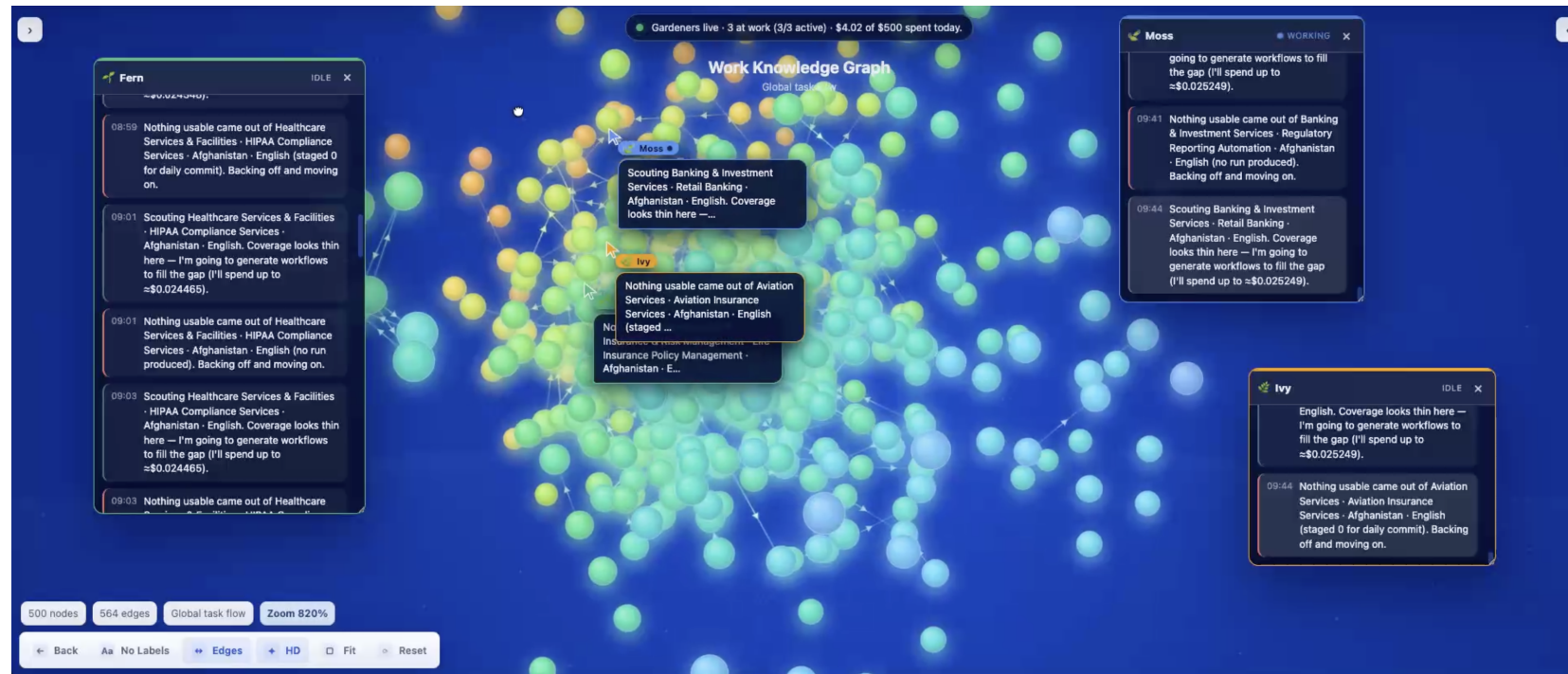
SEARCHING FOR WORK



Search identifies tasks to reuse when constructing a workflow.

The WKG supplies reusable work. Each workflow must still satisfy its own local requirements.

Gardeners repeatedly propose additions to the WKG



GARDENERS

AI agents that maintain and improve the WKG by identifying missing work, finding duplicates, consolidating tasks, and proposing edits, including additions.

Repeated proposals make admission a repeated decision: which additions can be committed?

Admission is a repeated decision

01 / PROPOSE

Retrieve + propose

Tasks, dependencies, and evidence arrive while the workflow is unfinished.

02 / VALIDATE

Validate + review

An addition may also close a branch or discharge an obligation.

03 / COMMIT

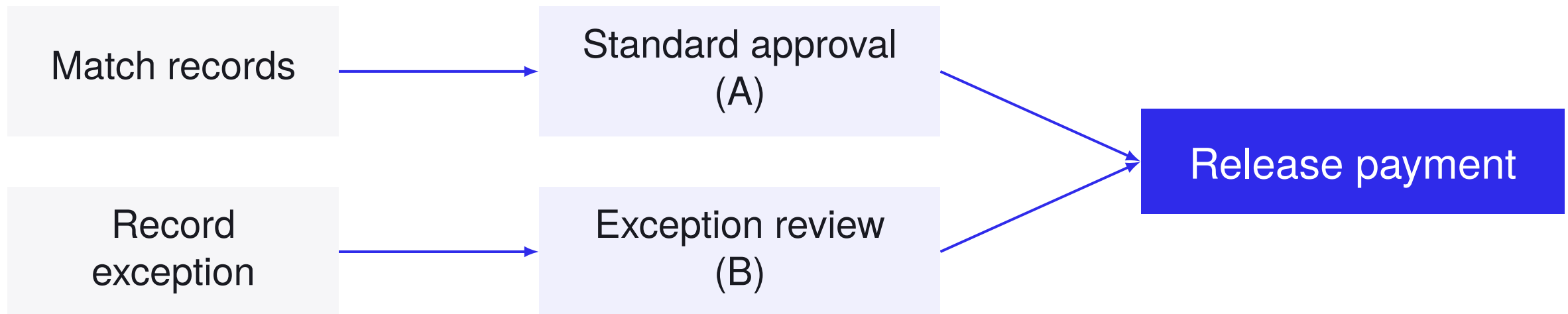
Commit snapshot

A legal edit may remove the last acceptable completion.

Admit an addition only if the updated workflow **still has an acceptable completion.**

Two routes can satisfy the payment policy

Construction goal: release payment through either standard approval or authorized exception review.



Either payment route gives an acceptable completion when its evidence and authorization requirements are met.

A reusable task may be unavailable in a local context

CONTEXT 1

Exception review
can be authorized.

A: standard approval

B: exception review

Both routes remain possible.

CONTEXT 2

Exception review
cannot be authorized.

A: standard approval

B: unavailable

Only standard approval remains possible.

A route is available only if its evidence and authorization requirements can still be met.

A locally legal edit can eliminate the last completion

Proposed addition: permanently close standard route A .

CURRENT STATE	AFTER CLOSING A	COMPLETION REMAINS?
A and B possible	Only B remains	Yes , through B
Only A possible	No route remains	No , dead end

Passing local checks does not guarantee that an acceptable completion remains.

Preserve at least one acceptable completion

Every admitted addition must leave a way to finish the workflow in finitely many further additions.

COMPLETION VIABILITY

Does the current state still admit an acceptable completion?

INCREMENTAL CHECKING

Carry evidence that a completion remains possible after each addition.

Completion viability means that an acceptable completion is possible, not that construction will eventually reach it.

The construction model

A construction state records the committed graph and all information needed to apply additions and determine whether completion remains possible.

S	Construction-state space, with states $s \in S$.
Σ	Addition alphabet: the set of additions a .
$\delta : S \times \Sigma \rightarrow S$	Deterministic transition to successor $\delta(s, a)$.
$F \subseteq S$	Accepting states: acceptable completed workflows.

An addition may modify graph structure, attach evidence, close a route, discharge an obligation, or mark completion.

An addition stream is a finite sequence of additions

Σ^* is the set of all finite sequences of elements of Σ , including the empty stream ε . We call each such sequence an **addition stream**.

CONCATENATION

uv

Concatenating u and v gives the stream u followed by the stream v .

PREFIX AND SUFFIX

$w = xu$

x is a prefix of w : its beginning.

u is the suffix of w after x . Either can be empty.

An addition stream specifies a sequence of additions, whether or not that sequence is admitted or completes the workflow.

The transition extends from additions to streams

Let $a \in \Sigma$ be one addition and $w \in \Sigma^*$ an addition stream. We also write a for the stream containing just that addition. The function $\delta^* : S \times \Sigma^* \rightarrow S$ gives the state reached after consuming a stream.

EMPTY STREAM

$$\delta^*(s, \varepsilon) = s$$

The empty stream leaves the state unchanged.

ONE ADDITION FOLLOWED BY A STREAM

$$\delta^*(s, aw) = \delta^*(\delta(s, a), w)$$

In aw , first apply addition a , then consume stream w from the successor state.

RESULT / STREAM COMPOSITION LEMMA

Consuming uv gives the same state as consuming u , then v .

$$\delta^*(s, uv) = \delta^*(\delta^*(s, u), v)$$

Completion viability

A construction state s is **viable** when some addition stream leads from s to an accepting state.

$$\text{Viable}(s) \iff \exists v \in \Sigma^* : \delta^*(s, v) \in F$$

ACCEPTING NOW

$s \in F$. The construction already satisfies the final policy.

VIABLE NOW

The construction may be unfinished, but at least one acceptable completion remains.

Result. Every accepting state is viable because consuming the empty stream leaves it in the accepting set F .

The viability residual

The **viability residual** of s is the set of addition streams that leave the construction viable after they are consumed.

$$R_s = \{u \in \Sigma^* : \exists v \in \Sigma^* \text{ such that } \delta^*(s, uv) \in F\}$$

Σ^* is the set of all finite addition streams, including the empty stream.

RESULT / RESIDUAL-SUCCESSOR EQUIVALENCE THEOREM

$$u \in R_s \iff \text{Viable}(\delta^*(s, u))$$

A stream u belongs to R_s when consuming u from state s leaves an acceptable completion possible.

Residual membership implies viable prefixes

PROPOSITION / PREFIX CLOSURE

If $w = xu$ and $w \in R_s$, then

$$x \in R_s$$

If wv reaches an accepting state, so does xuv , because $w = xu$.

COROLLARY / RESIDUAL NONEMPTINESS

A state is viable when its residual is nonempty.

$$\text{Viable}(s) \iff R_s \neq \emptyset$$

ε : the empty stream.

$\varepsilon \in R_s$: viable, not necessarily accepting.

$R_s = \emptyset$: no streams, no acceptable completion.

From a nonviable state, no addition stream can restore an acceptable completion.

Residuals update by left derivatives

A **language** $L \subseteq \Sigma^*$ is a set of addition streams. For $w \in \Sigma^*$, its **left derivative** by w is

$$w^{-1}L = \{u \in \Sigma^* : wu \in L\}$$

Keep the streams beginning with w , then remove that prefix. The notation w^{-1} is not a numerical inverse.

RESULT / DERIVATIVE LAW THEOREM

$$R_{\delta^*(s,w)} = w^{-1}R_s$$

After consuming w , a stream u belongs to the new residual if and only if wu belonged to the previous residual.

Closing standard approval changes the residual

In this finite example, a permanently closes route A. The one-addition streams A and B complete the workflow through standard approval and exception review, respectively.

BEFORE / BOTH ROUTES POSSIBLE

$$R_{AB} = \{\varepsilon, A, B, a, aB\}$$

ε is the empty stream. The stream aB closes route A, then completes the workflow through B.

BEFORE / ONLY STANDARD APPROVAL

$$R_{A\text{-only}} = \{\varepsilon, A\}$$

AFTER a / EXCEPTION REVIEW REMAINS

$$a^{-1}R_{AB} = \{\varepsilon, B\}$$

AFTER a / NO ROUTE REMAINS

$$a^{-1}R_{A\text{-only}} = \emptyset$$

In this example, closing A leaves completion through B possible in the first state, but removes the last acceptable completion in the second.

How can we check viability across additions?

- | | | |
|----|----------------------------|---|
| 01 | Check at completion | Does not check viability after each addition. |
| 02 | Keep one completion | Requires another acceptable completion if an addition rules out the chosen one. |
| 03 | Recompute viability | Check again from the full construction state after each addition. |
| 04 | Carry a certificate | Keep a certificate with the state and validate each update. |

We develop option 4: carry a certificate with the construction state and check its update before admitting each addition.

A certificate represents a set of addition streams

For a nonnegative integer i , let s_i be the state after i additions and c_i the **certificate** carried alongside it. The double brackets $\llbracket c_i \rrbracket$ denote the set of addition streams it represents, called its **denotation**.

SOUND AT THE CURRENT STATE

$$\llbracket c_i \rrbracket \subseteq R_{s_i}$$

Every stream represented by c_i belongs to the residual at state s_i .

NONEMPTY

$$\llbracket c_i \rrbracket \neq \emptyset$$

The certificate represents at least one addition stream. Soundness alone does not require this.

Result. A state s_i is viable if its certificate c_i is sound at s_i and represents at least one addition stream.

Checking a certificate update

Let a_i be the next addition. $\text{Check}(c_i, a_i, c_{i+1})$ means that the checker validates the update from c_i to c_{i+1} . A **stepwise-sound checker** requires both conditions below whenever it validates an update.

KEEP PREVIOUSLY SUPPORTED STREAMS

$$\llbracket c_{i+1} \rrbracket \subseteq a_i^{-1} \llbracket c_i \rrbracket$$

If c_{i+1} represents u , then c_i must already represent $a_i u$. This inclusion is called **refinement**.

RETAIN AT LEAST ONE STREAM

$$\llbracket c_{i+1} \rrbracket \neq \emptyset$$

The successor certificate must represent at least one addition stream.

A stepwise-sound checker requires the next certificate to represent at least one stream, and each such stream u must satisfy $a_i u \in \llbracket c_i \rrbracket$.

Certificates evolve with the construction

A **proof-carrying stream** is a finite sequence of admitted additions with certificates carried and checked at every step. A chain of n additions has the form

$$(s_0, c_0) \xrightarrow{a_0} (s_1, c_1) \xrightarrow{a_1} \dots \xrightarrow{a_{n-1}} (s_n, c_n)$$

PRODUCER

Proposes a_i , the successor certificate c_{i+1} , and checking evidence.

TRUSTED CHECKER

Validates the certificate update against c_i and a_i .

COMMIT

Only then advance both the certificate and the state, with $s_{i+1} = \delta(s_i, a_i)$.

Each admitted addition updates both the construction state and the certificate, after the checker validates their next step.

Soundness of proof-carrying streams

Start with a sound, nonempty certificate and admit only updates validated by a stepwise-sound checker. Then the construction is viable after every admitted prefix.

$$\emptyset \neq \llbracket c_i \rrbracket \subseteq R_{s_i}$$

The certificate c_i represents at least one stream, and every stream it represents belongs to the residual at state s_i .

PROOF STEP / AFTER ADDITION a_i

$$\llbracket c_{i+1} \rrbracket \subseteq a_i^{-1} \llbracket c_i \rrbracket \subseteq a_i^{-1} R_{s_i} = R_{s_{i+1}}$$

After consuming a_i , every stream represented by the next certificate belongs to the residual at the next state. The checker also requires at least one such stream.

A sound certificate can reject a viable addition

Both payment routes remain possible at s_i . The certificate c_i supports standard approval only. Let a_i permanently close standard approval.

Representing fewer streams may reduce certificate-maintenance work, but can rule out viable additions.

THE CONSTRUCTION

$$a_i^{-1} R_{s_i} \neq \emptyset$$

Exception review still provides an acceptable completion. The successor state is viable.

THE CERTIFICATE

$$a_i^{-1} \llbracket c_i \rrbracket = \emptyset$$

No supported stream survives this addition. The checker cannot validate a nonempty successor.

From a sound certificate, a stepwise-sound checker may reject a viable addition, but it cannot admit an addition leading to a nonviable state.

A sound certificate may represent only part of a residual

SOUND CERTIFICATE

$$\llbracket c_i \rrbracket \subseteq R_{s_i}$$

It may represent only **some** addition streams in R_{s_i} .

RESIDUALLY COMPLETE CERTIFICATE

$$\llbracket c_i \rrbracket = R_{s_i}$$

It represents **all** addition streams in R_{s_i} .

An update is **derivative-exact** when

$$\llbracket c_{i+1} \rrbracket = a_i^{-1} \llbracket c_i \rrbracket$$

A residually complete certificate represents every stream in the residual. A derivative-exact update represents every stream in the previous certificate's derivative.

A proof state retains information from the construction

A certificate may support only some viable addition streams. We now ask what information a **proof state** must retain to classify viability correctly after every addition stream.

ENCODING

$$E : S \rightarrow P \quad p = E(s)$$

P is the proof-state space. The encoding E maps construction state s to proof state p .

PROOF-STATE UPDATE

$$U : P \times \Sigma \rightarrow P$$

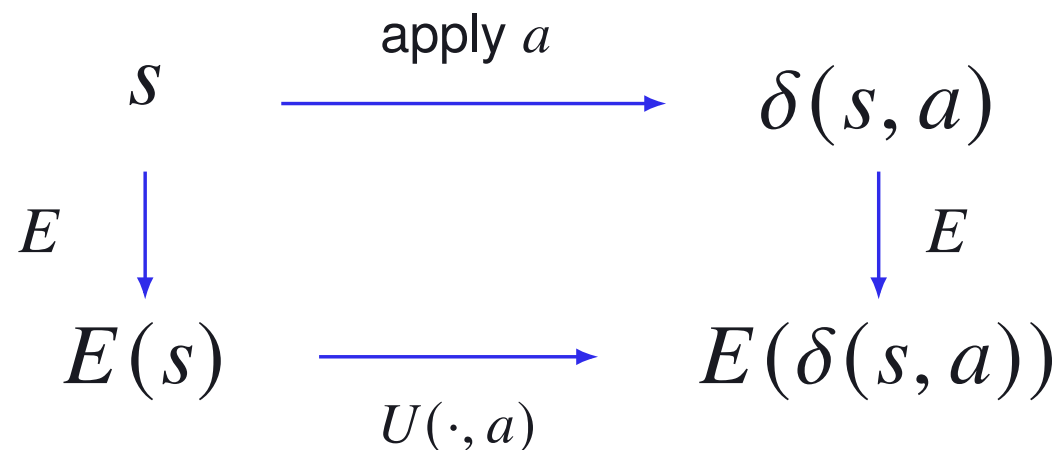
$U(p, a)$ computes the next proof state using only the current proof state p and addition a .

The goal is to update viability information through the proof state without inspecting the full WKG construction state at every step.

Updates and viability must agree with the construction

Let Live be the set of proof states classified as viable, with $\text{Live} \subseteq P$. The abstraction (P, E, U, Live) is **viability-exact** when both conditions hold for every $s \in S$ and $a \in \Sigma$.

UPDATING COMMUTES WITH ENCODING



CLASSIFICATION AGREES WITH VIABILITY

$$E(s) \in \text{Live} \iff \text{Viable}(s)$$

A proof state is in Live if and only if the construction state it encodes is viable.

We take $P = E(S)$: every proof state represents a construction state.

Updating $E(s)$ by addition a gives the same proof state as applying a to s and then encoding the successor.

Equal proof states imply equal viability residuals

In a viability-exact abstraction, starting from the same proof state and consuming the same addition stream gives the same viability classification.

TWO VIABLE STATES CAN DIFFER

State s : both approval routes

State t : standard approval only

Closing standard approval leaves s viable but t nonviable. Hence $E(s) \neq E(t)$.

RESIDUAL SEPARATION

$$E(s) = E(t) \implies R_s = R_t$$

Equal encodings give the same viability result after every addition stream, so their residuals are equal.

Viability-exact proof states must distinguish construction states whenever the same addition stream can leave one viable and the other nonviable.

Group construction states with the same residual

Two construction states are **residual equivalent** when they have the same viability residual. We write this relation as $s \equiv_R t$.

RESIDUAL-EQUIVALENCE CLASS

$$[s] = \{t \in S : R_t = R_s\}$$

$[s]$ is the group of all construction states with the same residual as s .

QUOTIENT SPACE

$$Q = S / \equiv_R = \{[s] : s \in S\}$$

The quotient Q contains these groups. Encoding s as $[s]$ gives one proof state per distinct residual.

Result. States in the same class give the same viability answer after every addition stream, so they can share one proof state.

The residual quotient keeps the fewest distinctions

Use $[s]$ as the proof state. Equal residuals have equal derivatives, so the update below does not depend on which state represents the class.

DEFINITION / UPDATE ON CLASSES

Define $U_Q : Q \times \Sigma \rightarrow Q$ by

$$U_Q([s], a) = [\delta(s, a)]$$

DEFINITION / LIVE CLASSES

Define $\text{Live}_Q \subseteq Q$ by

$$[s] \in \text{Live}_Q \iff R_s \neq \emptyset$$

These rules give a viability-exact abstraction. States with equal residuals share a proof state, while states with different residuals must have different proof states.

The quotient retains only the distinctions needed to determine viability after every addition stream. This makes it the **coarsest** viability-exact abstraction.

When finitely many proof states suffice

A finite viability-exact abstraction exists if and only if there are finitely many distinct residuals.

$$\min\{|P| : (P, E, U, \text{Live}) \text{ is viability-exact}\} = |Q|$$

FINITELY MANY RESIDUALS

One proof state per residual class suffices, even if the construction-state space is infinite.

INFINITELY MANY RESIDUALS

Finitely many proof states cannot give correct viability answers after every addition stream.

This criterion tells us whether finitely many proof states can track viability, and the minimum number needed.

Classical results support a changing proof obligation

- | | | |
|----|---|---|
| 01 | Derivatives and Nerode equivalence | Derivatives update residuals. Under a Nerode-style equivalence, construction states are grouped so that no common future viability test can distinguish them. |
| 02 | Proof-carrying systems and plans | Separate proof production from trusted checking. |
| 03 | Workflow soundness and supervisory control | Supply the completion and nonblocking perspective. |

These results give conditions for carrying evidence of workflow viability and identify the distinctions that viability-exact proof states must retain.

Conclusion

A sound, nonempty initial certificate and stepwise-sound checked updates preserve completion viability throughout construction.

- | | | |
|----|-------------------------------------|--|
| 01 | Keep completion possible | After each admitted addition, at least one acceptable completion remains possible. |
| 02 | Update the certificate | Keep a nonempty set of addition streams from the previous certificate's derivative. |
| 03 | Retain residual distinctions | Viability-exact abstractions must separate construction states with different residuals. |

For WKG construction, the aim is to update the current certificate after each admitted addition so that it continues to show that an acceptable completion remains possible.

Questions?

Proof-Carrying Streamed Workflow Graphs

Preserving Completion Viability with Evolving Certificates

Phillip Kingston · Théo Fagnoni

phillip.kingston@opus.com

1st Conference and Workshop on Mathematics and Artificial Intelligence
Nazarbayev University, Astana · 9–11 September 2026

Lean checks a defined part of the argument

FORMALIZED IN LEAN 4

Residual laws and viability after admitted additions

Certificate-stream soundness

Residual separation

Residual-language update and viability classification

PROVED IN THE PAPER ONLY

Residual quotient and factorization

Finite-state minimality

Realization of arbitrary prefix-closed residual languages

Artifact: `code/ProofCarryingStreams.lean` (Lean's `Init` library only)

Every viability-exact abstraction maps onto the quotient

For any viability-exact abstraction (P, E, U, Live) , define $\phi : P \rightarrow Q$ by

$$\phi(E(s)) = [s]$$

Equal encodings have equal residual classes, so this is well defined. Since $P = E(S)$, the rule determines ϕ uniquely and reaches every class.

PRESERVES UPDATES

$$\phi(U(p, a)) = U_Q(\phi(p), a)$$

PRESERVES VIABILITY

$$p \in \text{Live} \iff \phi(p) \in \text{Live}_Q$$

Updating then mapping to the quotient gives the same class as mapping first then updating. The viability answer is unchanged.

Every prefix-closed language can be a residual

Take any prefix-closed language $L \subseteq \Sigma^*$. Use streams as construction states and append one addition at each step.

$$S = \Sigma^* \quad \delta(w, a) = wa \quad s_0 = \varepsilon \quad F = L$$

$$u \in R_{s_0} \iff \exists v \in \Sigma^* : uv \in L \iff u \in L$$

If $uv \in L$, prefix closure gives $u \in L$. If $u \in L$, choose $v = \varepsilon$. Hence $R_{s_0} = L$, including when L is empty.

The construction model alone does not guarantee that a residual is finite, decidable or compactly represented.

Apply certificate checks to workflows and WKG updates

	OPUS WORKFLOW CONSTRUCTION	WKG TASK-VERSION UPDATE
State s	Supplier-payment graph, required evidence and authorized reviewers.	Draft task-catalog version and a fixed set of dependent workflows.
Accepting set F	Payment-release workflow completed with the required evidence and approvals.	Published catalog version leaves every named workflow completable.
Addition a	Close standard approval while keeping exception review available.	Retire task v1 and bind its named dependents to task v2.
Next certificate	Certify a stream adding exception evidence, approval and payment release.	Certify a migration stream with compatible data mappings and approvals.
Check	Require the previous certificate to support a followed by that stream.	Apply the same check. Reject if a named workflow has no acceptable completion.

Given a sound, nonempty initial certificate, the same checked-update rule keeps completion possible in each model.